Darren Lee
CSE13S

4/14/21
Functions
- Just as assignment statements are not the same as equations in high school Algebra, functions in C are not the same as mathematical functions
    - May or may not return a value
    - May or may not have side-effects
- In other languages, we might call them procedures or subroutines
- Java aficionados call them methods
    - Please, do not do that when writing in C

How is it different
- In math, a function f maps an element of a set called the domain onto a set called the range
- There are many ways to define a function
- In C, we call function what in other languages might be called procedure or subroutine
    - Return a value of the function type

What is a function in programming
- A function is a block of code that performs a certain task
    - It gives a name to code that hopefully performs a logically consistent task
- Functions are defined exactly once
- Functions must be declared before they are used, can declare as many times as you want
- Programs can declare and call a function as many times as desired
- main()
    - Is a special function
    - Is run when program starts
    - All other functions are subordinate to main()

Loader takes code from program and puts in in memory then does the function call to main

Why do we like functions?
- should
    - Define abstractions that are consistent and make sense logically
    - Give names to those sequences of code
    - Hide the implementation
- We can use them to:
    - Refactor repeated sequences of code
    - Simplify the code to aid understanding
- Functions should never be:
    - Arbitrary sequences of statements

Function definition
- Function head

        - Return_type
            - Defines the type of function's return value
            - Return type may be void or any object type (except array type_
        - function_name()
            - Function's name
        - Parameters
            - Contained in comma-separated list of declarations
            - If function has no parameters, can put void

Return values

- Value may be void, no value
- Can return any scalar value
- Can return a struct (please don't)
- Can return a pointer

Function naming

- Same naming rules as variables
- Can't:
    - Start with a number or any punctuation other than _ or $
    - Use the same name as another function
        - There are no nested functions in C
- For this class, we will be using Snake case
    - My_function_name

parameters/arguments

- In math, when we have a function like f(x) and we write f(2) we substitute 2 for x
    - In programming languages, this is called call-by-name, and is rare
    - C supports this as textual substitution in macros with the C Preprocessor
- Most programming languages use either call-by-value, call-by-reference or both
- C uses call-by-value, except for arrays, and only because of their relation to pointers

Parameters

- Formal parameters
    - This is the name of the parameter as it is used inside of the function body
- Actual parameters
- Call-by-value means a copy of the actual parameter is placed in the formal parameter
    - This is the only method supported by C

Call-by-value

- Arguments passed into a function are copied
    - Any changes made to the parameters inside the function has no effect

Call-by-reference

- The references of the arguments passed into a function are copied meaning any changes made to the parameters inside the function has an effect on the actual values

- Instead of passing values to the called function, references to the original variables are passed
- C does not use call-by-reference
    - But you can accomplish it by passing a pointer

swap()

- C does not have true call-by-reference, so we use pointers
    - addresses, instead of values, are passed as arguments

Function prototypes

- Provides compiler with desc of functions that will be used later in the program
    - Functions in programs can't be called unless they have either:
        - Been declared and defined

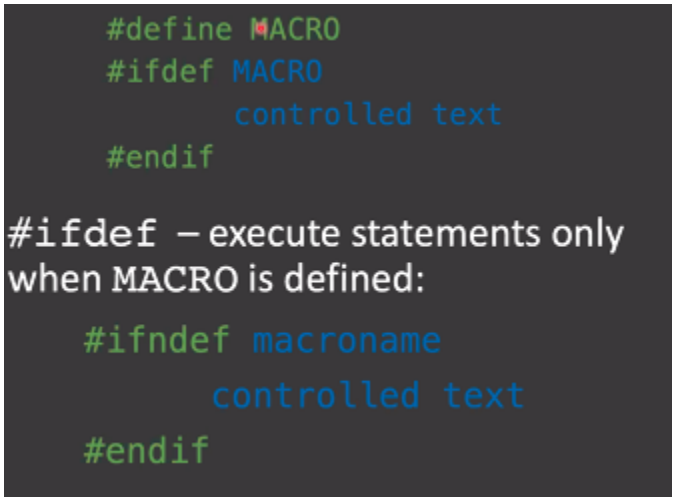
#include

- Preprocessor directive
    - Before compilation, C source files are processed by preprocessor
    - Preprocessor is a macro processor to transform programs before compilation
    - Macros in C operate through text replacement
- Pastes code of given file into current file
- Used to include functions defined in other libraries

#define

- Preprocessor directive that defines a macro for the program
- C preprocessor performs all text replacement for defined macros prior to compilation
- The compiler never sees what is defined, diff from constant

Conditional directives

- Set of preprocessor directives that uses conditional statements to include code selectively
    - Uses value of conditions valuated during compilation
- #ifndef-execute statements when MACRO is undefined:
- 

Header files

- Should only contain things that are shared between source files:
  - Function declarations
  - Macro definitions
  - Data structure and enumeration definitions
  - Global variables (see coding standard for proper usage)
  - Any #include directives required to compile
- Uses the file extension .h

Example Header File

```
# ifndef _STACK_H
# define _STACK_H
# include <stdint.h>
# include <stdbool.h>

typedef uint32_t item;

typedef struct stack
{
        uint32_t size;
        uint32_t top;
        item *entries;
} stack;

# define MIN_STACK 128
# define INVALID   0xDeadD00d

stack *newStack();

void delStack(stack *);

item pop(stack *);

void push(stack *, item);

bool empty(stack *);
# endif
```

New types

Function interfaces

- For stack abstract data type
- Note that everything is contained within the header guard

Some standard header files

- Stdio.h for input.output
- Inttypes.h for fixed width integer types
- Time.h for date/time utilities
- Stdbool.h for boolean types
- Ctype.h for function to determine the type contained in character data

Extern

- Extends the visibility of variables and functions such that they can be called by any program file, provided that the declaration is known
- Functions in C are implicitly prepended by extern typically used for global variables
- Extern variables are declared outside of functions
- Available until end of the execution of the function

Extern counter

```c
#include "extern.h"

int counter = 42; // Global counter definition.

void decrement(void) {
  counter--;
  return;
}

void increment(void) {
  counter++;
  return;
}
```

```c
#ifndef __EXTERN_H__
#define __EXTERN_H__

extern int counter; // Counter declaration in extern.c.

void decrement(void); // Function prototype for decrement().

void increment(void); // Function prototype for increment().

#endif
```

Using < > means in standard place “ “ means in the file path

Static

- Can be declared inside and outside a function
- Declared inside a function if the value of the variable needs to persist across function calls
- Declared outside a function if the value of the variable needs to be accessed by multiple functions

Recursion

- A function may call itself! We will discuss this in detail later
- Syntax of recursive functions:

-
```c
void function_name(){
  function_name(); // function calls itself
}
```
- Recursive function must always define exit conditions to prevent the function from being called an unbounded number of times
- Recursive code is more compact and may be easier to understand

Summary
- Functions provide the ease of running a sequence of code repeatedly
- Functions can return void, scalar values, pointers, and structs; cannot return arrays
- For this class only use snake case function naming
- Formal parameters are used inside the body of a functions

4/16/21
What is an array?
- Homogeneous collection of elements
  - All elements have the same type
- Arrays can have one dimension
  - Often called a vector
- Arrays can have two dimensions
  - Often called a matrix
  - C treats a matrix as an array

Arrays are ordered
- In math , we write a vector as a-hat=<a1,...,an>
- In C, arrays start at zero: a[0],...,a[n-1]
  - Why? It's easier, since a[0] is the base address of the array
- A[i] comes before a[i+1] in memory
  - What about a[n]
    - It's past the end of the array, don't do that
- A[i] comes after a[i-1] in memory
  - Does a[-2] make sense?
    - No, it's before the base of the array

Declaring an array
```c
int a[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}; // The first 10 positive integers

int b[10]; // Ten empty slots (usually zero, depending on where declared)

float c[] = {3.1416, 2.7183, 0.57722, 1.6180}; // Four important constants
```
- If you have an initialization list, then you don't need a count
- If you have a count, then you don't need an initialization list
  - But you can have one, of course

- If the list is too short, then C will fill the rest with zero

In Memory

The assembly language shows us what's really happening

```
        .section        __DATA,__data
        .globl  _a                      ## @a
        .p2align        4
_a:
        .long   1                       ## 0x1
        .long   2                       ## 0x2
        .long   3                       ## 0x3
        .long   4                       ## 0x4
        .long   5                       ## 0x5
        .long   6                       ## 0x6
        .long   7                       ## 0x7
        .long   8                       ## 0x8
        .long   9                       ## 0x9
        .long   10                      ## 0xa

        .globl  _c                      ## @c
        .p2align        4
_c:
        .long   1078530041              ## float 3.14159989
        .long   1076754593              ## float 2.7183001
        .long   1058260145              ## float 0.577220023
        .long   1070537376              ## float 1.61800003

        .comm   _b,40,4                 ## @b
```

Matrices

- In C, we write: type m[3][3];
  - The elements are m[0][0] to m[2][2]
- C stores the array in row major order
  - That means one row in memory, then the next row, then the next row
- FORTRAN uses column major order

In memory

- Memory is one dimensional
- Rows are laid out sequentially

```
int m[2][2] = {{1, 2}, {3, 4}};
```

```
        .section        __DATA,__data
        .globl  _m                      ## @m
        .p2align        4
_m:
        .long   1                       ## 0x1
        .long   2                       ## 0x2
        .long   3                       ## 0x3
        .long   4                       ## 0x4
```

-

Matrix multiplication

```
void matMul(int n, int m, int p, float a[n][m], float b[m][p], float c[n][p]) {
  for (int i = 0; i < n; i += 1) {
    for (int j = 0; j < p; j += 1) {
      c[i][j] = 0.0;
      for (int k = 0; k < m; k += 1) {
        c[i][j] += a[i][k] * b[k][j];
      }
    }
  }
  return;
}
```

How are we changing c[][]

- Arrays are the exception to the rule that parameters in C are always passed by value
- Why is this?
    - Arrays can be large, so you don't want to copy them onto the stack
    - Arrays and pointers are intimately related
- The name of the array is just a pointer to element zero of the array
- *a is the same as a[0] *(2)a is the same as a[2]

& (address of) operator
& gives the memory location (address) of a variable

```c
#include <inttypes.h>
#include <stdio.h>

int main(void) {
  float x[2] = {0.1, 3.1415};

  uint64_t x0 = (uint64_t) &x[0];
  uint64_t x1 = (uint64_t) &x[1];

  printf("x[0] is at %" PRIu64 " and x[1] is at %" PRIu64 "\n", x0, x1);
  return 0;
}
```

```
x[0] is at 140732850817744 and x[1] is at 140732850817748
```

Sizeof operator

- The sizeof operator tells us the number of bytes used by a variable
- Works on arrays, structures, unions, when the compiler can know how much memory is used

```c
#include <inttypes.h>
#include <stdio.h>

int main(void) {
  double x[] = {1.6180, 2.7183, 0.57722, 3.1416};

  uint64_t xs = sizeof(x);
  uint64_t x1 = sizeof(&x[1]);

  printf("&x takes %" PRIu64 " bytes and x takes %" PRIu64
         " bytes and x[1] takes %" PRIu64 " bytes\n",
         (uint64_t)sizeof(&x), xs, x1);
  return 0;
}
```

```
&x takes 8 bytes and x takes 32 bytes and x[1] takes 8 bytes
```

In C, if you give sizeof an array, and it knows the number of bytes at compile time, it will give the size of the actual array

Use strlen() function for strings

C does not stop you when you reach the end of the array, it lets you keep going to the thing after the array in memory or before the array in memory

C doesn't know the size of the array because pointers and arrays the are the same thing
Don't allocate memory on top of each other

C will allocate to avoid overlap

```c
#include <stdio.h>

int main(void) {
  char *s = "Hello world";
  char t[] = "Goodbye cruel world";

  printf("sizeof(s) = %lu, sizeof(*s) = %lu\n", sizeof(s), sizeof(*s));
  printf("sizeof(t) = %lu, sizeof(*t) = %lu\n", sizeof(t), sizeof(*t));
  return 0;
}
```

```
sizeof(s) = 8, sizeof(*s) = 1
sizeof(t) = 20, sizeof(*t) = 1
```

- `sizeof` when applied to an array gives the size of the array
- `sizeof` when applied to a pointer gives the size of the pointer

Arrays and pointers in C are related in fundamental and often confusing ways
For a single dimensional array,
a[i]==*(a+i)
A is the address of a[0]
A[i] is the array slot that is at a+i*sizeof(a[0])
Pointers automatically do the multiplication by sizeof

```c
#include <stdio.h>
#include <stdlib.h>

int *newArray(int elements) { return (int *)calloc(elements, sizeof(int)); }

int main(void) {
    int *a;
    int n;
    do {
        printf("n > 1: ");
        scanf("%d", &n);
    } while (n < 2);
    a = newArray(n);
    for (int i = 0; i < n; i += 1) {
        a[i] = i * i;
    }
    for (int i = 0; i < n; i += 1) {
        printf("%d^2 = %d\n", i, a[i]);
    }
    free(a);
    return 0;
}
```

Arrays and pointers are equivalent in C

Malloc is similar to calloc except it does not initialize memory to 0; they dynamically allocate memory on the heap

Malloc just leaves whatever was in memory to begin with, you don't know what it is

```c
int **makeMatrix(int n, int m) {
  int **t = calloc(n, sizeof(int *)); // Allocate a column of row pointers
  for (int i = 0; i < n; i += 1) {
    t[i] = calloc(m, sizeof(int)); // Allocate each row
  }
  return t;
}
```

A matrix is an array of pointers

What is a string?

- In C, a string is an array of characters that ends in '\0'
- It can be written as:
    - Char s[]="Hello world"
    - Char *s="Goodbye cruel world!"
    - `char s[] = {'L', 'e', 'g', 'a', 'l', 0}`
- Most people use the *s version
- A 100 character (99 printable) empty string is
    - Char s[100]

- A string is just an array of characters that we have agreed is terminated by a null (zero) character

Assignment and copying

```
char *s = "Wally Wonder Badger";
char *t;

t = s;
```

- What does that mean?
- It means that s and t point to (reference) the same memory location
- It does not make a copy of s

What is going on here?

```
#include <stdio.h>

int main(void) {
  int x[] = {1, 2, 3, 4};
  int *y;
  y = x;
  for (int i = 0; i < 4; i += 1) printf("%d ", x[i]); printf("\n");
  y[2] = 17;
  for (int i = 0; i < 4; i += 1) printf("%d ", x[i]); printf("\n");
  return 0;
}
```

We'll use integers for simplicity
Y is an alias for x

- They reference a place in memory

strcmp()

```
int strcmp(char s[], char t[]) {
  for (int i = 0; s[i] && t[i]; i += 1) {
    if (s[i] - t[i]) {
      return s[i] - t[i];
    }
  }
  return 0;
}
```

- As long as both characters are not null, subtract them
- <0 means s<t

- >0 means s>t
- =0 means s==t

strlen()

```
int strlen(char s[]) {
  int length = 0;
  while (s[length]) {
    length += 1;
  }
  return length;
}
```

- As long as s[length] is not null, move on to the next one
- Length is the number of non-null characters

strcpy()

```
void strcpy(char s[], char t[]) {
  for (int i = 0; t[i]; i += 1) {
    s[i] = t[i];
  }
  s[strlen(t)] = '\0'; // null terminate
  return;
}
```

- As long as t[i] is not null, copy it to s[i]
- And be sure to tack a null on the end

You should never write code like that!

- Why not?
  - Did you check the array bounds?
- This is the source of most buffer-overflow attacks!

Safer version
strncmp()

```c
int strncmp(char s[], char t[], int n) {
  for (int i = 0; i < n && s[i] && t[i]; i += 1) {
    if (s[i] - t[i]) {
      return s[i] - t[i];
    }
  }
  return 0;
}
```

- We just have to add a quick check to make sure we haven't gone too far
- It depends on you knowing how large the arrays are, of course
- Note that n must account for the null at the end